

09-small-models-road-to-the-top-part-2

May 26, 2025

```
[1]: # install fastkaggle if not available
try: import fastkaggle
except ModuleNotFoundError:
    !pip install -Uq fastkaggle

from fastkaggle import *
```

This is part 2 of the [Road to the Top](#) series, in which I show the process I used to tackle the [Paddy Doctor](#) competition, leading to four 1st place submissions. If you haven't already, first check out [part 1](#).

0.1 Going faster

First we'll repeat the steps we used last time to access the data and ensure all the latest libraries are installed:

```
[2]: comp = 'paddy-disease-classification'
path = setup_comp(comp, install='fastai "timm>=0.6.2.dev0"')
from fastai.vision.all import *
set_seed(42)
```

A big issue I noticed last time was that originally I created the notebook on my home PC, and each epoch of the resnet we created took under 20 seconds to run. But on Kaggle they took over 3 minutes each! Whilst Kaggle's GPUs are less powerful than what I've got at home, that doesn't come close to explaining this vast difference in speed.

I noticed when Kaggle was running that the "GPU" indicator in the top right was nearly empty, and the "CPU" one was always full. This strongly suggests that the problem was that Kaggle's notebook was CPU bound by decoding and resizing the images. This is a common problem on machines with poor CPU performance – and indeed Kaggle only provides 2 virtual CPUs at the time of writing.

We really need to fix this, since we need to be able to iterate much more quickly. What we can do is to simply resize all the images to half their height and width – which reduces their number of pixels 4x. This should mean an around 4x increase in performance for training small models.

Luckily, fastai has a function which does exactly this, whilst maintaining the folder structure of the data: `resize_images`.

```
[3]: trn_path = Path('sml')
```

```
[4]: resize_images(path/'train_images', dest=trn_path, max_size=256, recurse=True)
```

This will give us 192x256px images. Let's take a look:

```
[5]: dls = ImageDataLoaders.from_folder(trn_path, valid_pct=0.2, seed=42,
    item_tfms=Resize((256,192)))

dls.show_batch(max_n=3)
```



In this notebook, we'll be experimenting with a few different architectures and image processing approaches (item and batch transforms). In order to make this easier, we'll put our modeling steps together into a little function which we can pass the architecture, item transforms, and batch transforms to:

```
[6]: def train(arch, item, batch, epochs=5):
    dls = ImageDataLoaders.from_folder(trn_path, seed=42, valid_pct=0.2,
    ↪item_tfms=item, batch_tfms=batch)
    learn = vision_learner(dls, arch, metrics=error_rate).to_fp16()
    learn.fine_tune(epochs, 0.01)
    return learn
```

Our `item_tfms` already resize our images to small sizes, so this shouldn't impact the accuracy of our models much, if at all. Let's re-run our `resnet26d` to test.

```
[7]: learn = train('resnet26d', item=Resize(192),
    batch=aug_transforms(size=128, min_scale=0.75))
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

That’s a big improvement in speed, and the accuracy looks fine.

0.2 A ConvNeXt model

I noticed that the GPU usage bar in Kaggle was still nearly empty, so we’re still CPU bound. That means we should be able to use a more capable model with little if any speed impact. Let’s look again at the options in [The best vision models for fine-tuning](#). `convnext_small` tops the performance/accuracy tradeoff score there, so let’s give it a go!

```
[8]: arch = 'convnext_small_in22k'
```

```
[9]: learn = train(arch, item=Resize(192, method='squish'),  
                  batch=aug_transforms(size=128, min_scale=0.75))
```

```
/home/vscode/.local/lib/python3.10/site-packages/timm/models/_factory.py:114:  
UserWarning: Mapping deprecated model name convnext_small_in22k to current  
convnext_small.fb_in22k.  
    model = create_fn(  

```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

Wow our error rate has halved! That’s a great result. And, as expected, the speed hasn’t gone up much at all. This seems like a great model for iterating on.

0.3 Preprocessing experiments

So, what shall we try first? One thing which can make a difference is whether we “squish” a rectangular image into a square shape by changing it’s aspect ratio, or randomly crop out a square from it, or whether we add black padding to the edges to make it a square. In the previous version we “squished”. Let’s try “crop” instead, which is fastai’s default:

```
[10]: learn = train(arch, item=Resize(192),  
                  batch=aug_transforms(size=128, min_scale=0.75))
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

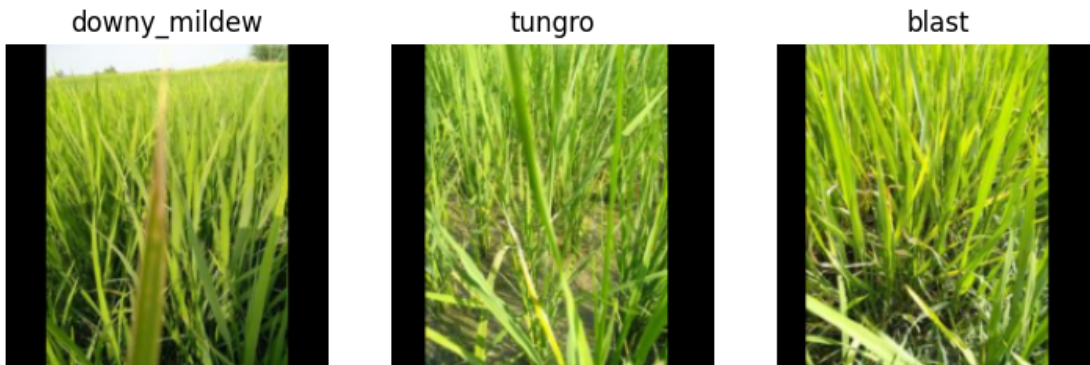
```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

That doesn’t seem to have made much difference...

We can also try padding, which keeps all the original image without transforming it – here’s what that looks like:

```
[11]: dls = ImageDataLoaders.from_folder(trn_path, valid_pct=0.2, seed=42,
      item_tfms=Resize(192, method=ResizeMethod.Pad, pad_mode=PadMode.Zeros))
      dls.show_batch(max_n=3)
```



```
[12]: learn = train(arch, item=Resize((256,192), method=ResizeMethod.Pad,
      ↪pad_mode=PadMode.Zeros),
      batch=aug_transforms(size=(171,128), min_scale=0.75))
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

That's looking like a pretty good improvement.

0.4 Test time augmentation

To make the predictions even better, we can try [test time augmentation](#) (TTA), which [our book](#) defines as:

During inference or validation, creating multiple versions of each image, using data augmentation, and then taking the average or maximum of the predictions for each augmented version of the image.

Before trying that out, we'll first see how to check the predictions and error rate of our model without TTA:

```
[13]: valid = learn.dls.valid
      preds,targs = learn.get_preds(dl=valid)
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

```
[14]: error_rate(preds, targs)
```

```
[14]: TensorBase(0.0375)
```

That's the same error rate we saw at the end of training, above, so we know that we're doing that correctly.

Here's what our data augmentation is doing – if you look carefully, you can see that each image is a bit lighter or darker, sometimes flipped, zoomed, rotated, warped, and/or zoomed:

```
[15]: learn.dls.train.show_batch(max_n=6, unique=True)
```

bacterial_leaf_streak



bacterial_leaf_streak



bacterial_leaf_streak



bacterial_leaf_streak



bacterial_leaf_streak



bacterial_leaf_streak



If we call `tta()` then we'll get the average of predictions made for multiple different augmented versions of each image, along with the unaugmented original:

```
[16]: tta_preds, _ = learn.tta(dl=valid)
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Let's check the error rate of this:

```
[17]: error_rate(tta_preds, targs)
```

```
[17]: TensorBase(0.0341)
```

That's a huge improvement! We'll definitely want to use this for any submission we make!

0.5 Scaling up

Now that we've got a pretty good model and preprocessing approach, let's scale it up to larger images and more epochs. We'll switch back our path to the original un-resized images, and use 12 epochs using our best settings so far, with larger final augmented images:

```
[18]: trn_path = path/'train_images'
```

```
[19]: learn = train(arch, epochs=12,
                  item=Resize((480, 360), method=ResizeMethod.Pad, pad_mode=PadMode.
↪Zeros),
                  batch=aug_transforms(size=(256,192), min_scale=0.75))
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

This is around twice as accurate as our previous best model - let's see how it performs with TTA too:

```
[20]: tta_preds, targs = learn.tta(dl=learn.dls.valid)
      error_rate(tta_preds, targs)
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
[20]: TensorBase(0.0288)
```

Once again, we get a big boost from TTA. This is one of the most under-appreciated deep learning tricks, in my opinion! (I'm not sure there's any other frameworks that make it quite so easy, so perhaps that's part of the reason why...)

0.6 Submission

We're now ready to get our Kaggle submission sorted. First, we'll grab the test set like we did in the last notebook:

```
[21]: tst_files = get_image_files(path/'test_images').sorted()
      tst_dl = learn.dls.test_dl(tst_files)
```

Next, do TTA on that test set:

```
[22]: preds,_ = learn.tta(dl=tst_dl)
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

We need to indices of the largest probability prediction in each row, since that's the index of the predicted disease. `argmax` in PyTorch gives us exactly that:

```
[23]: idxs = preds.argmax(dim=1)
```

Now we need to look up those indices in the `vocab`. Last time we did that using `pandas`, although since then I realised there's an even easier way!:

```
[24]: vocab = np.array(learn.dls.vocab)
      results = pd.Series(vocab[idxs], name="idxs")
```

```
[25]: ss = pd.read_csv(path/'sample_submission.csv')
      ss['label'] = results
      ss.to_csv('subm.csv', index=False)
      !head subm.csv
```

```
image_id,label
200001.jpg,hispa
200002.jpg,normal
200003.jpg,blast
200004.jpg,blast
200005.jpg,blast
200006.jpg,brown_spot
200007.jpg,dead_heart
200008.jpg,brown_spot
200009.jpg,hispa
```

```
[26]: if not iskaggle:
      from kaggle import api
      api.competition_submit_cli('subm.csv', 'convnext small 256x192 12 epochs_
      ↳tta', comp)
```

Warning: Looks like you're using an outdated API Version, please consider updating (server 1.7.4.2 / client 1.5.16)

100%| | 70.5k/70.5k [00:01<00:00, 62.7kB/s]

This gets a score of 0.9827, which is well within the top 25% of the competition – that’s a big improvement, and we’re still using a single small model!

```
[27]: # This is what I use to push my notebook from my home PC to Kaggle

if not iskaggle:
    push_notebook('isaacjohanziebarth', 'small-models-road-to-the-top-part-2',
                  title='Small models: Road to the Top, Part 2',
                  file='09-small-models-road-to-the-top-part-2.ipynb',
                  competition=comp, private=True, gpu=True)
```

Kernel version 1 successfully pushed. Please check progress at
<https://www.kaggle.com/code/isaacjohanziebarth/small-models-road-to-the-top-part-2>

0.7 Conclusion

We’ve made a big step today, despite just using a single model that trains in under 20 minutes even on Kaggle’s rather under-powered machines. Next time, we’ll try scaling up to some bigger models and doing some ensembling.

If you found this notebook useful, please remember to click the little up-arrow at the top to upvote it, since I like to know when people have found my work useful, and it helps others find it too. And if you have any questions or comments, please pop them below – I read every comment I receive!